Volcano for the Agentic AI Era:

Unified Scheduling Across Training, Inference, and Agents

Ricardo Aravena · Rainie Li

KubeCon + CloudNativeCon North America 2026
Salt Lake City · November 9–12

Speakers

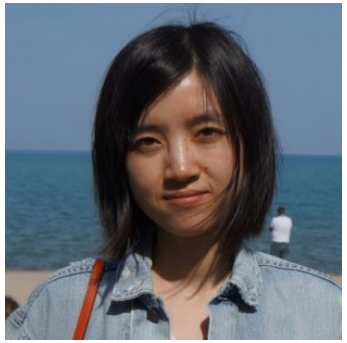


Ricardo Aravena

AI Infrastructure Lead · Nunchux AI

CNCF TOC Alternate (GB)
Cloud native and AI infrastructure

nunchux.ai · CNCF TOC



Rainie Li

Pinterest

Data processing infrastructure
Apache YuniKorn PMC member

Apache YuniKorn · rainieli

01

Unified scheduling

Different paths. Shared infrastructure.

01 Unified scheduling

02 Multi-level gang & network topology

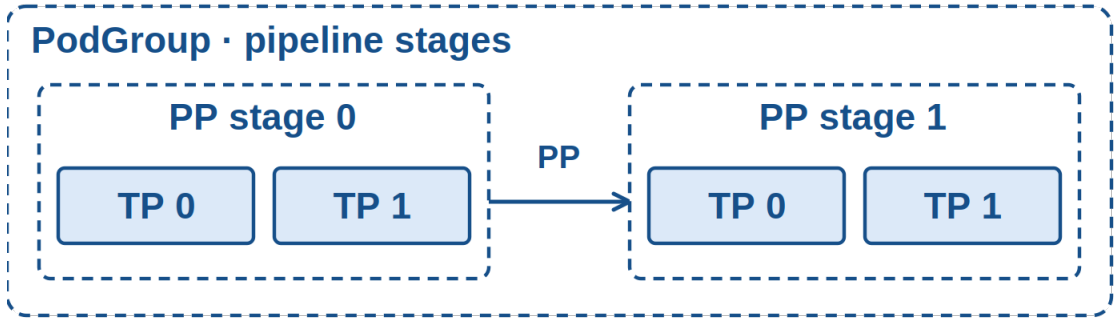
03 Ecosystem

04 Benchmark framework & progress

Different Workloads, Shared Infrastructure

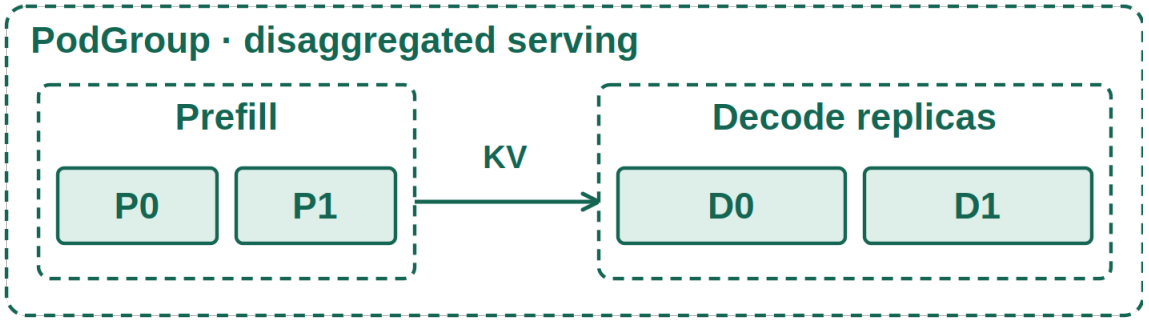
Training, inference, and agent applications put different demands on the same cluster.

Training



Complete TP / PP gangs + network locality

Inference



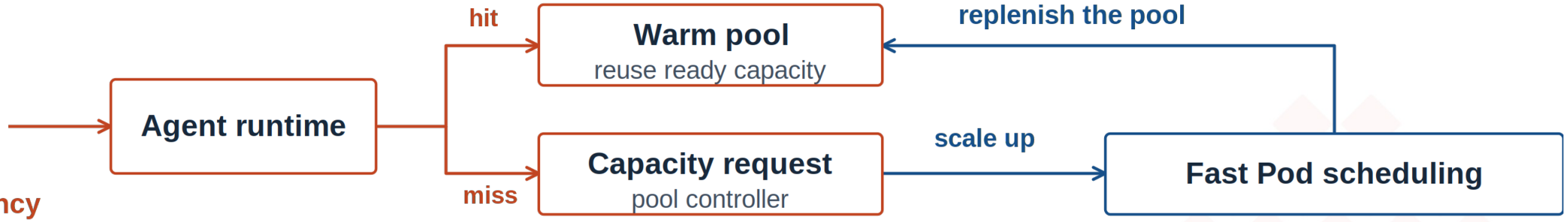
Role groups + topology-aware placement

Agents

Bursty arrivals

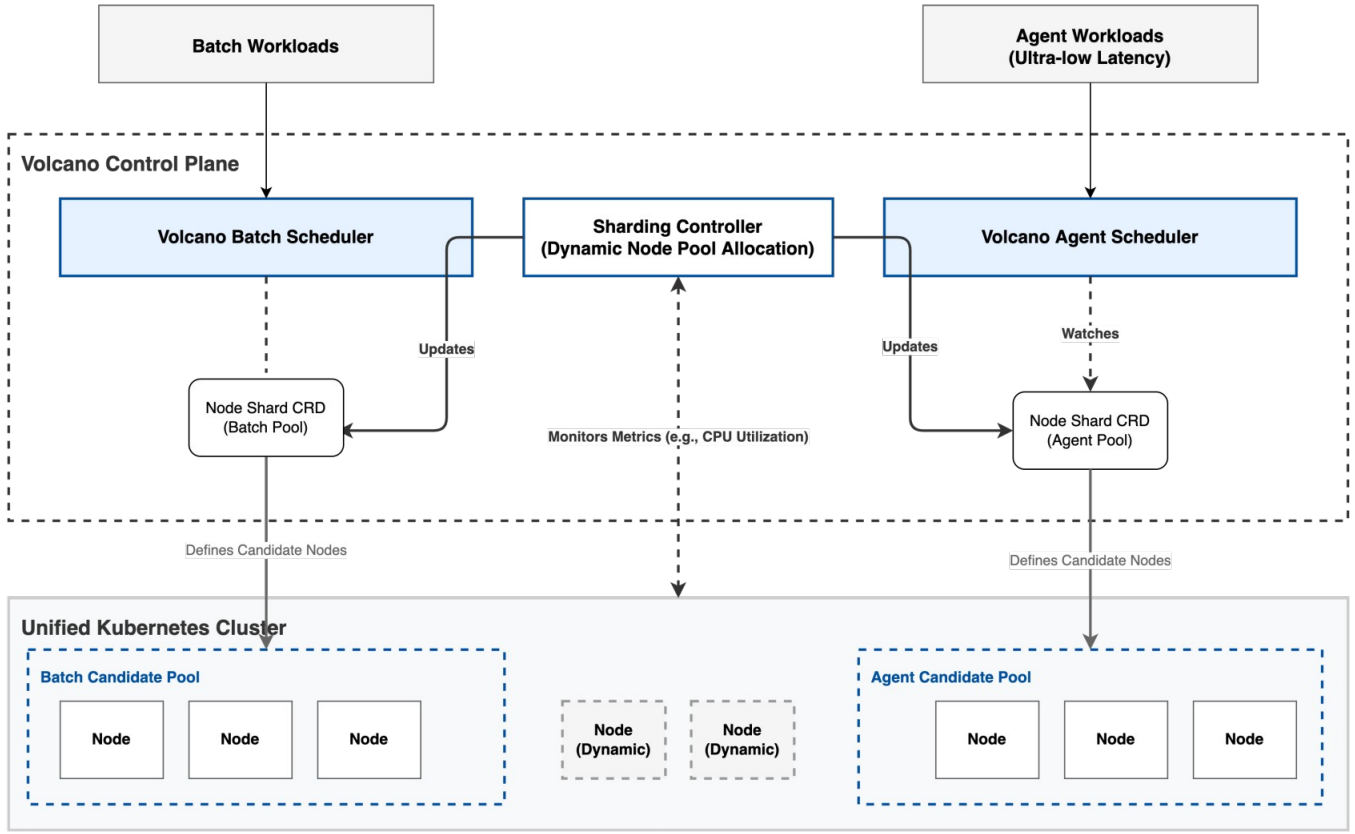


Low startup latency



A Coordinated Scheduling Framework

Specialized scheduling paths coordinate through NodeShard.



01 Workload paths

Batch-oriented scheduling and a Pod-oriented fast path.

02 Local coordination

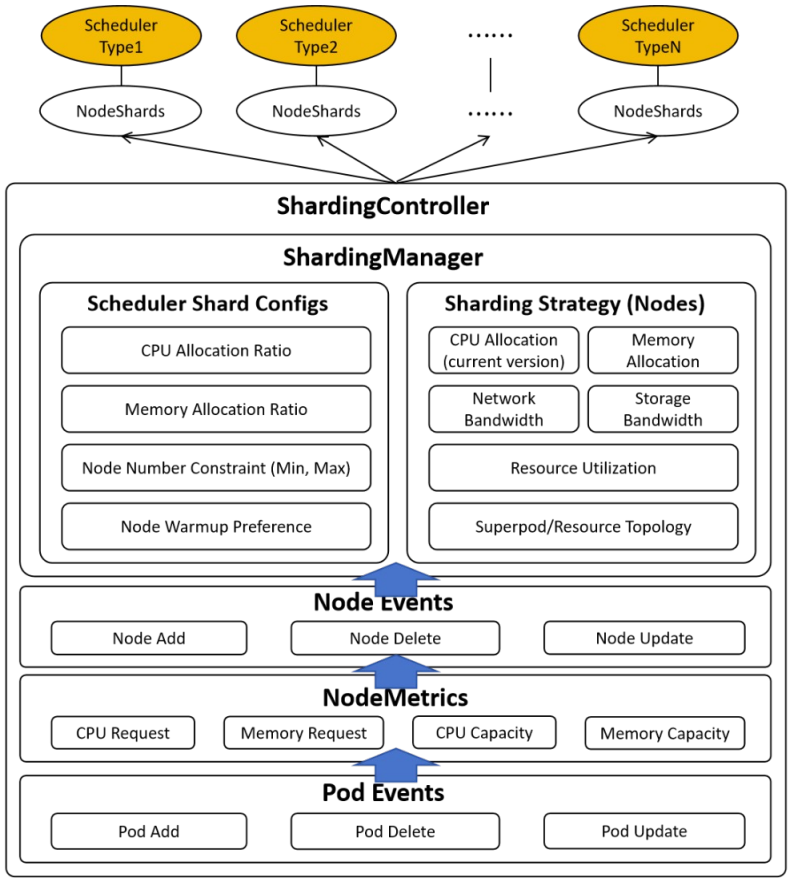
Parallel workers use Binder conflict checks.

03 Shared node access

NodeShard coordinates candidate-node pools.

NodeShard: Desired Nodes and Current Use

Changing a shard coordinates future placement. It does not migrate running Pods.



NodeShard · spec / status excerpt

```
spec:
  nodesDesired:
  - node1
  - node2
status:
  nodesInUse:
  - node0
  - node1
  nodesToRemove:
  - node0
  nodesToAdd:
  - node2
```

Desired membership ≠ immediate use

Coordinators reconcile node access.

02

Multi-level gang & network topology

From available nodes to runnable workloads.

01 Unified scheduling

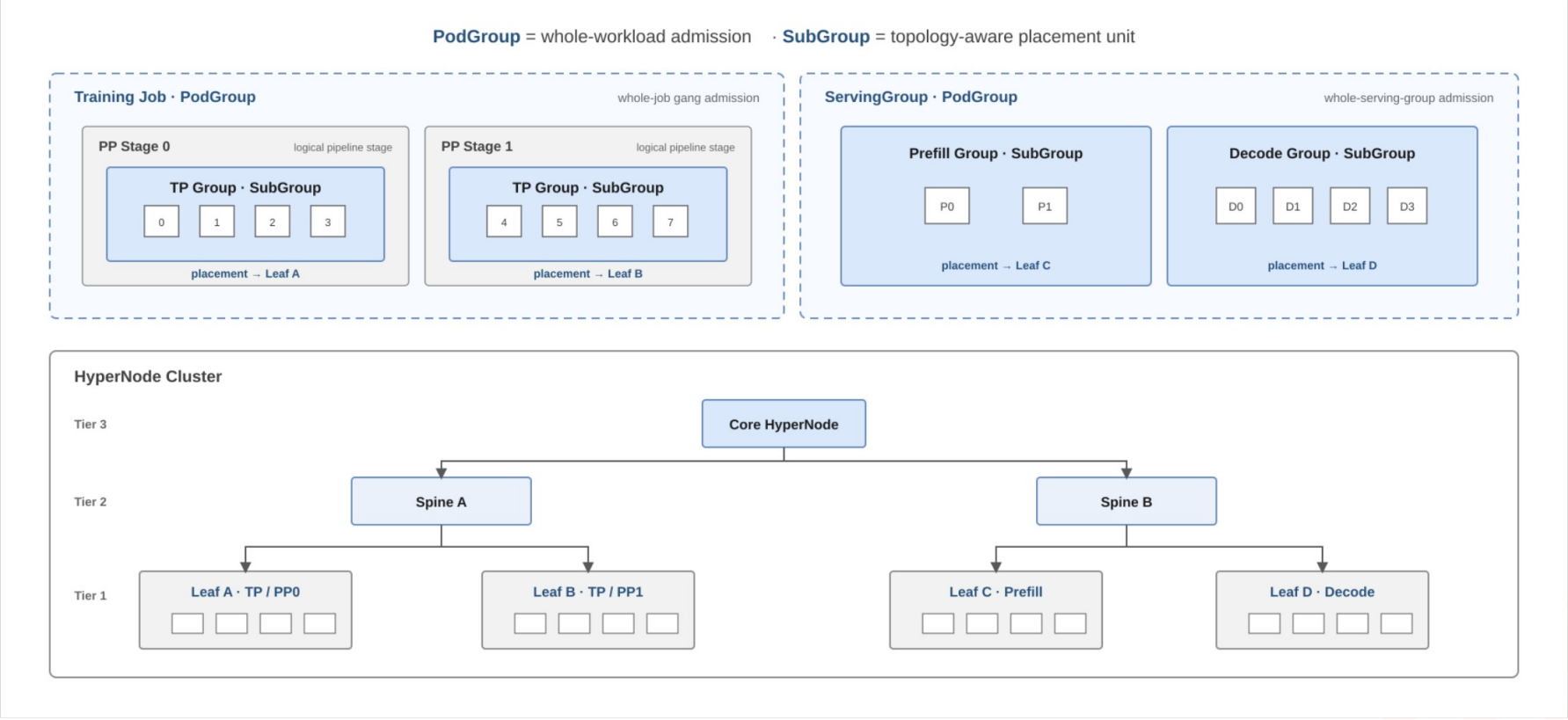
02 Multi-level gang & network topology

03 Ecosystem

04 Benchmark framework & progress

PodGroup and SubGroup: Preserve Workload Structure

Gang requirements apply to the workload and to its complete subgroups.

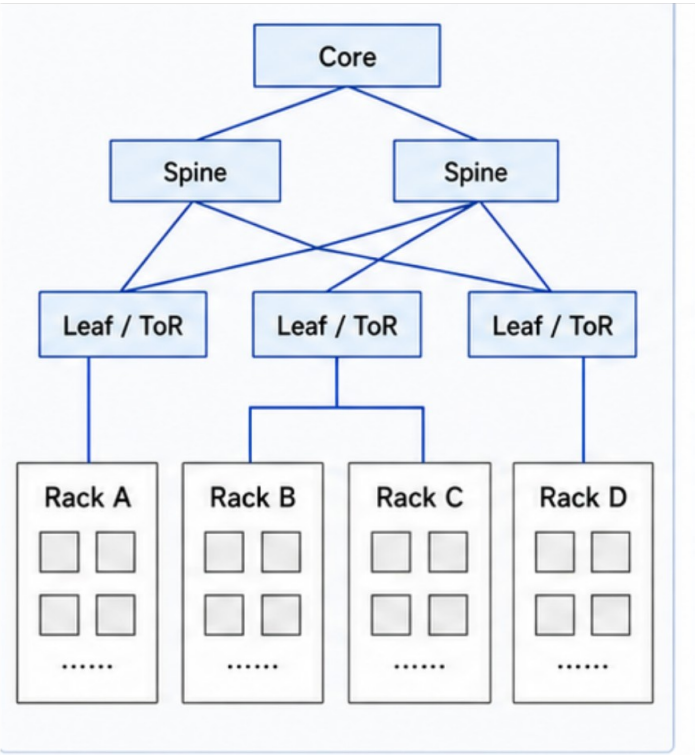


A SubGroup policy selects Pods. Replica identity can split them into multiple SubGroups.

HyperNode: From Physical Fabric to Scheduling Domains

A parent HyperNode groups child HyperNodes. A leaf groups Kubernetes Nodes.

Physical fabric



Parent · HyperNode.spec

```
tier: 2
tierName: spine
members:
- type: HyperNode
  selector:
    exactMatch:
      name: rack-a
```

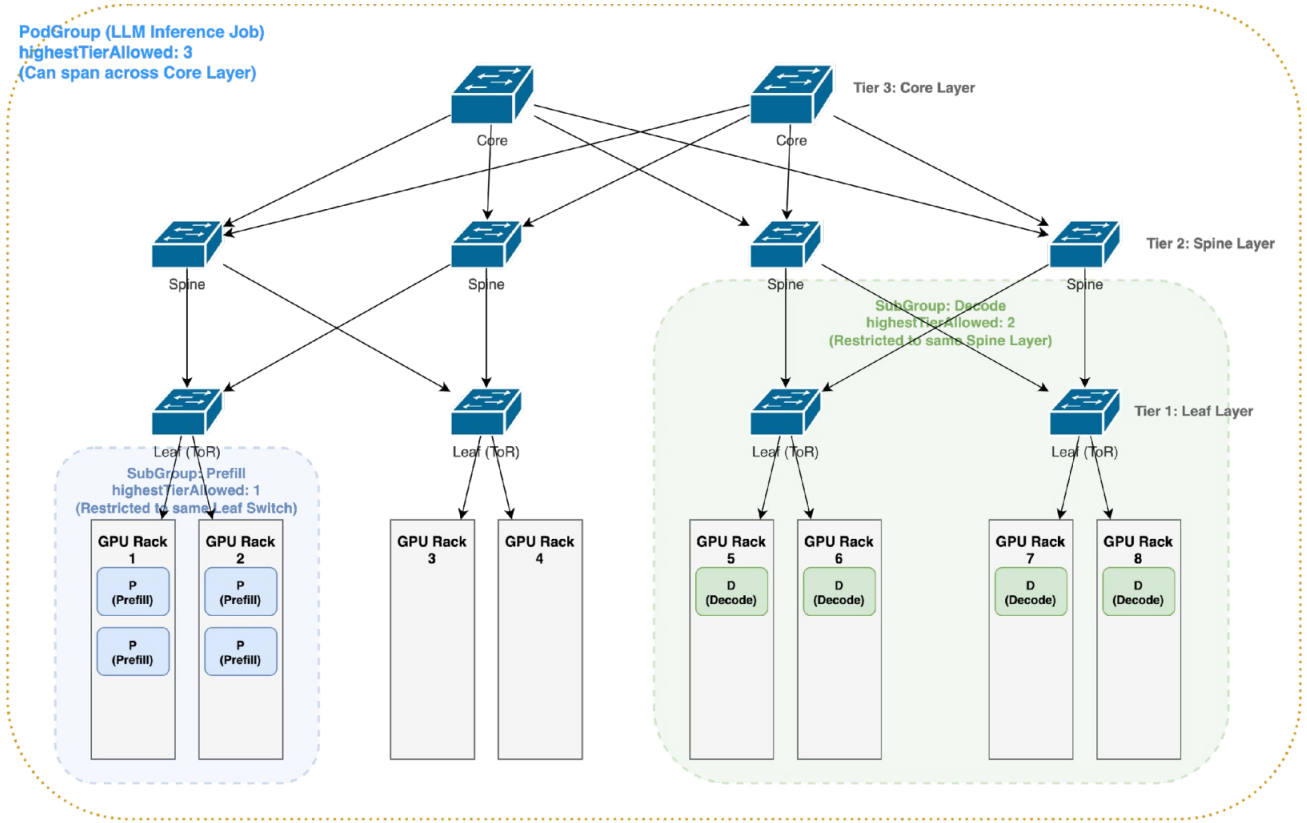
Leaf · HyperNode.spec

```
tier: 1
tierName: leaf
members:
- type: Node
  selector:
    labelMatch:
      matchLabels:
        rack: rack-a
```

The same hierarchy is available through the standalone HyperNode component.

Place Groups at Different Topology Levels

An eight-Pod instance spans a larger domain. Prefill and Decode use different locality constraints.



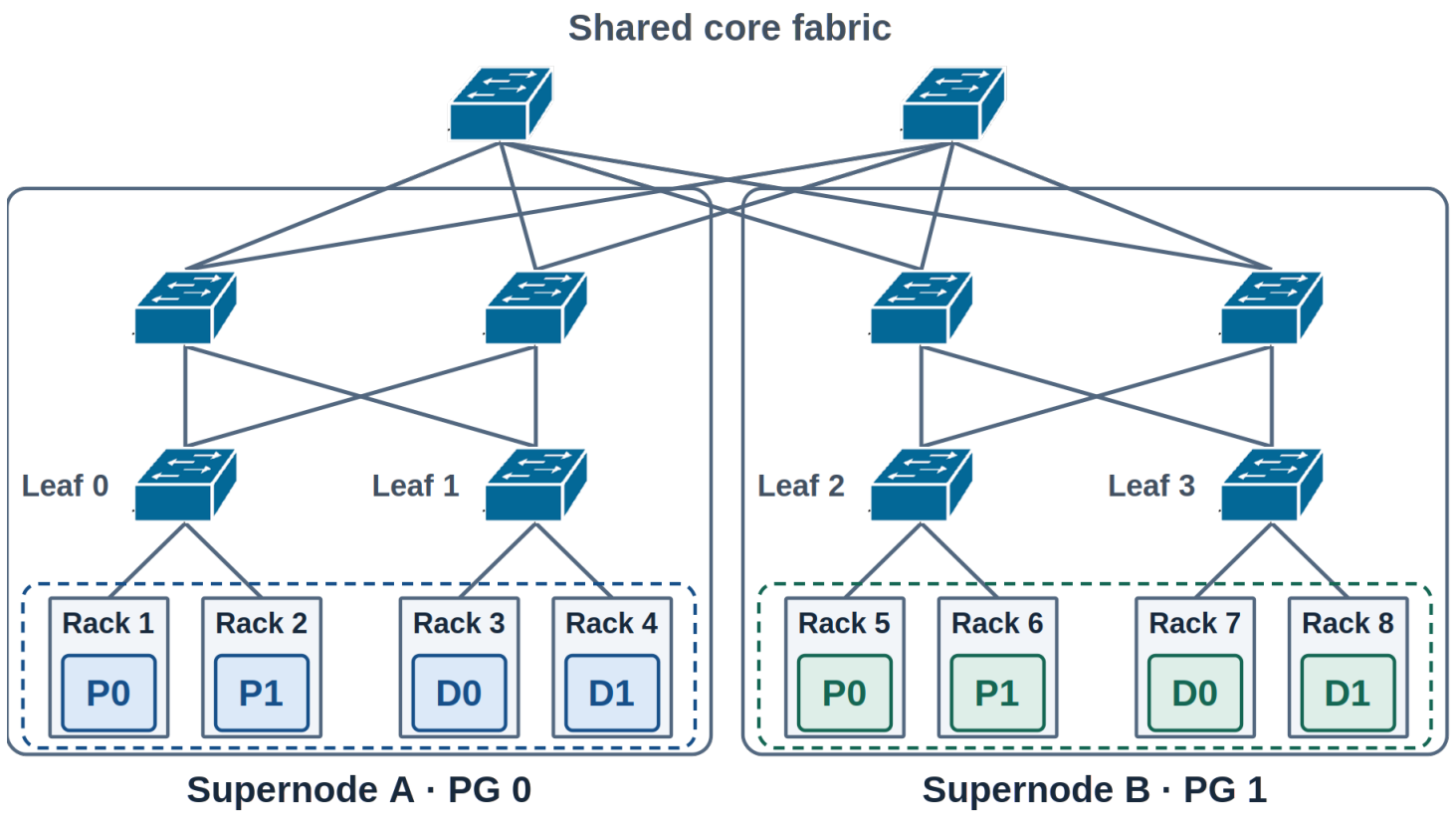
Named tiers: core (3) · spine (2) · leaf (1)

PodGroup.spec · key fields

```
minMember: 8
networkTopology:
  mode: hard
  highestTierName: core
subGroupPolicy:
- name: prefill
  subGroupSize: 4
  networkTopology:
    mode: hard
    highestTierName: leaf
- name: decode
  subGroupSize: 4
  networkTopology:
    mode: hard
    highestTierName: spine
```

Keep Each Instance Together, Place Instances Apart

ServingGroup → PodGroup: a Kthena scenario. Anti-affinity integration in Kthena is proposed.



PodGroup.spec · key fields

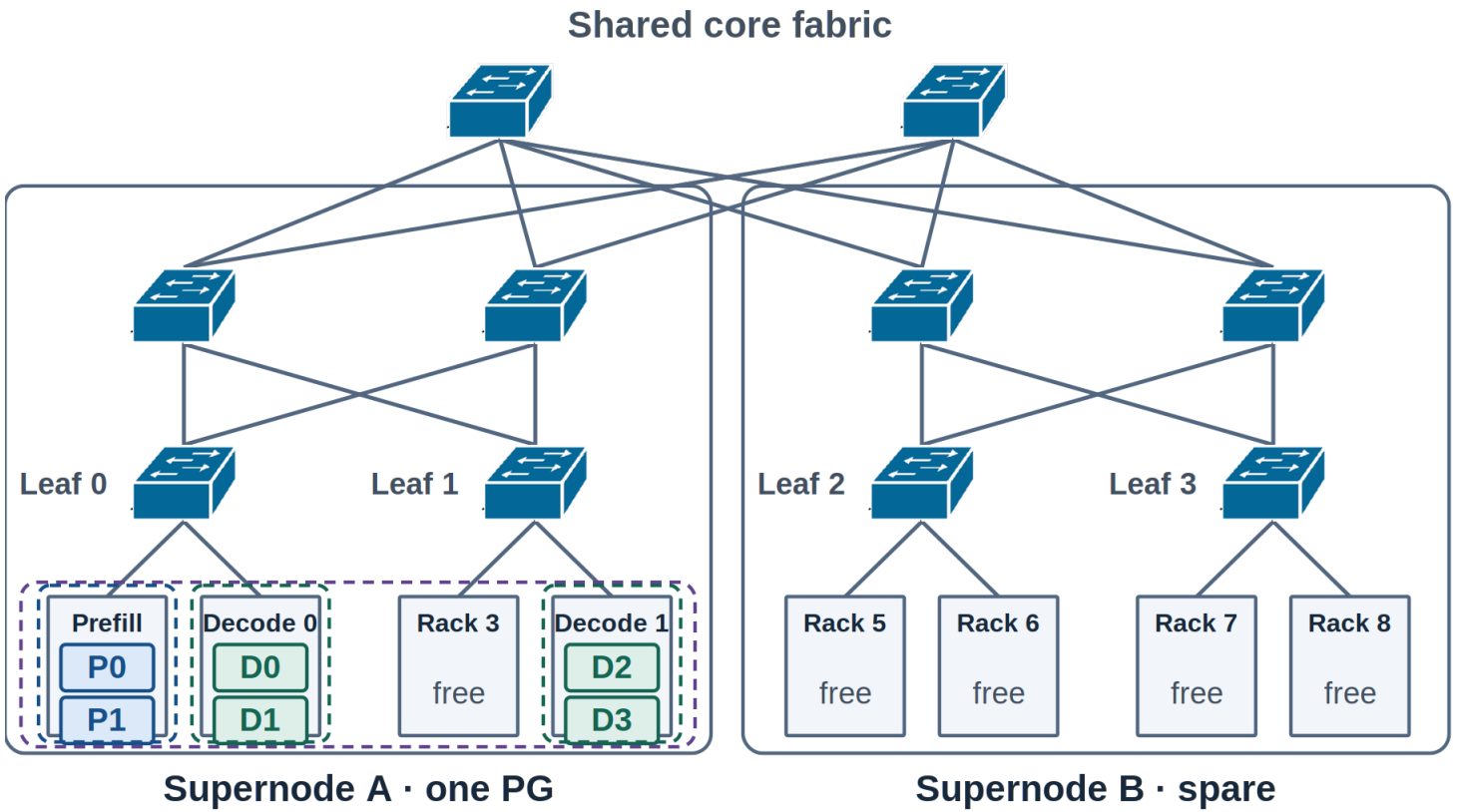
```
networkTopology:
  mode: hard
  highestTierName: supernode
topologyAffinity:
  podGroupAntiAffinity:
    required:
    - podGroupSelector:
        matchLabels:
          app: model-a
      topologyTierName: supernode
```

Each PodGroup: app=model-a

One domain can fail without taking both complete instances with it.

Next: Relationships Between SubGroups

Keep cooperating groups local while separating replicas inside the same PodGroup.



Proposed SubGroup relationships

Affinity · supernode

Prefill + Decode

Keep KV exchange inside the same supernode.

Anti-affinity · leaf

Decode replica 0 + replica 1

Keep complete replicas on separate leaf domains.

03

Ecosystem

Workload controllers carry scheduling intent.

01 Unified scheduling

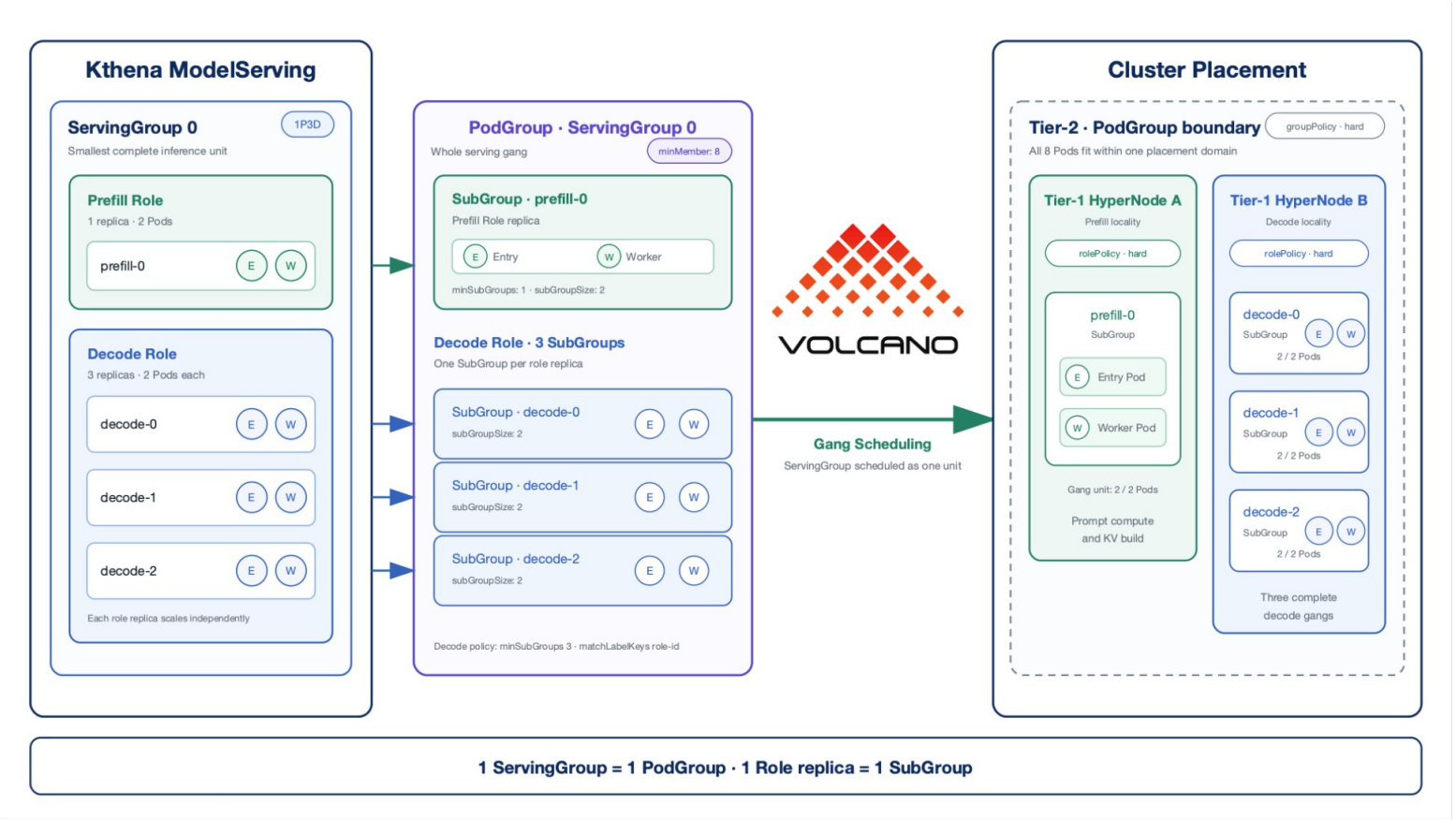
02 Multi-level gang & network topology

03 Ecosystem

04 Benchmark framework & progress

Kthena: From Serving Roles to Scheduling Groups

One ServingGroup maps to one PodGroup. Each role replica forms a SubGroup.



ModelServing.spec · excerpt

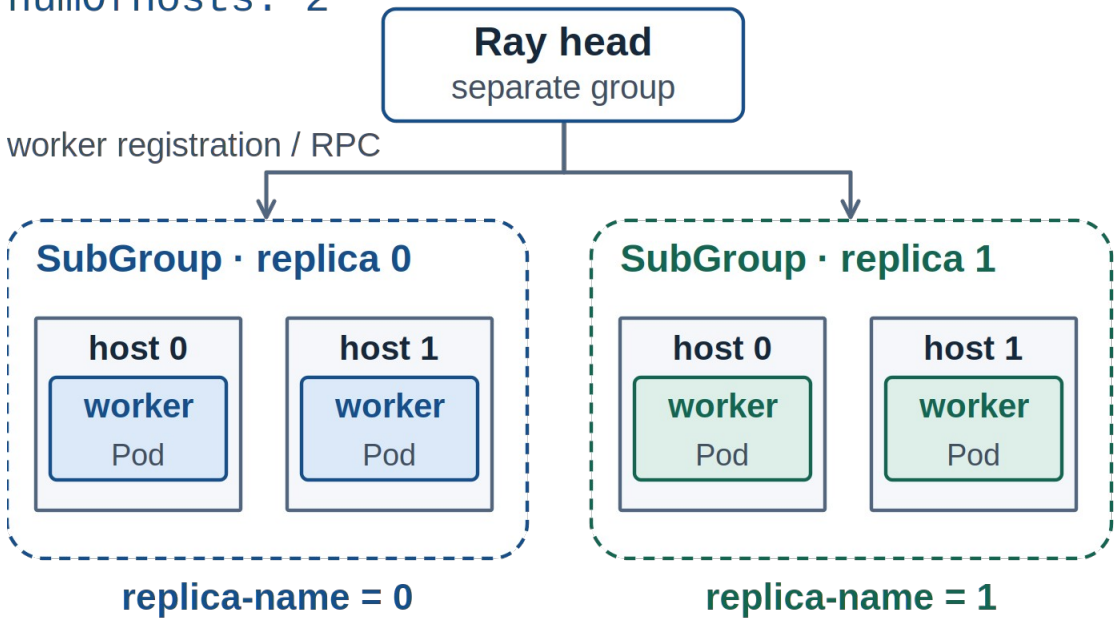
```
schedulerName: volcano
replicas: 1
template:
  gangPolicy:
    minRoleReplicas:
      prefill: 1
      decode: 3
  roles:
  - name: prefill
    replicas: 1
    workerReplicas: 1
  - name: decode
    replicas: 3
    workerReplicas: 1
```

KubeRay: A Replica Can Span Multiple Pods

The replica label separates complete groups inside one WorkerGroup.

RayCluster · worker group

```
minReplicas: 2
numOfHosts: 2
```



PodGroup.spec · generated grouping

```
subGroupPolicy:
- name: worker
  labelSelector:
    matchLabels:
      ray.io/group: worker
  matchLabelKeys:
  - ray.io/worker-group-replica-name
  minSubGroups: 2
  subGroupSize: 2
```

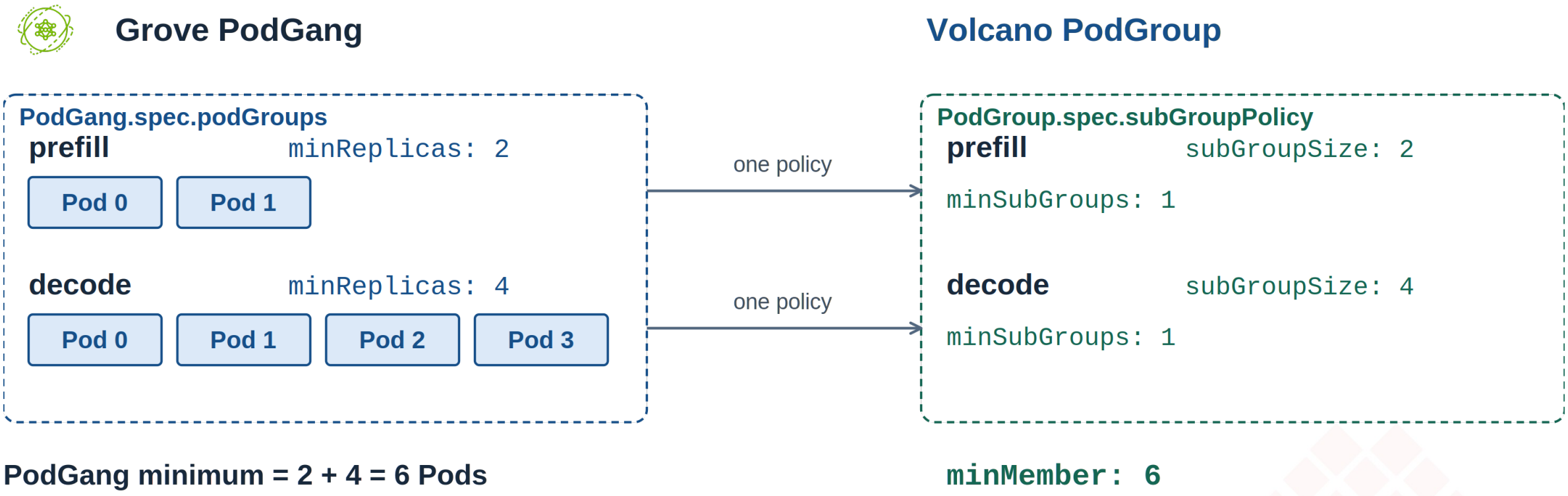
One replica label → one complete SubGroup

Two replicas × two hosts = four worker Pods

Gang support is merged. Replica topology is in progress.

Grove: Carry PodGang Structure into Volcano

The backend translates Grove PodGroups into Volcano SubGroup policies.



Gang and SubGroup mapping are available. Topology support is pending.

Next: RBG, Kubeflow, and LWS

Build on current integrations. Carry finer group structure and placement requirements.



RBG

In progress

RoleInstance → SubGroup

Extend the existing gang integration.



Kubeflow

Planned

Training groups + topology

Build on the Volcano scheduler plugin.



LWS

Planned

Multi-role / multi-group support

Connect group relationships to scheduling.

04

Benchmark framework & progress

Measure behavior across workload shapes.

01 Unified scheduling

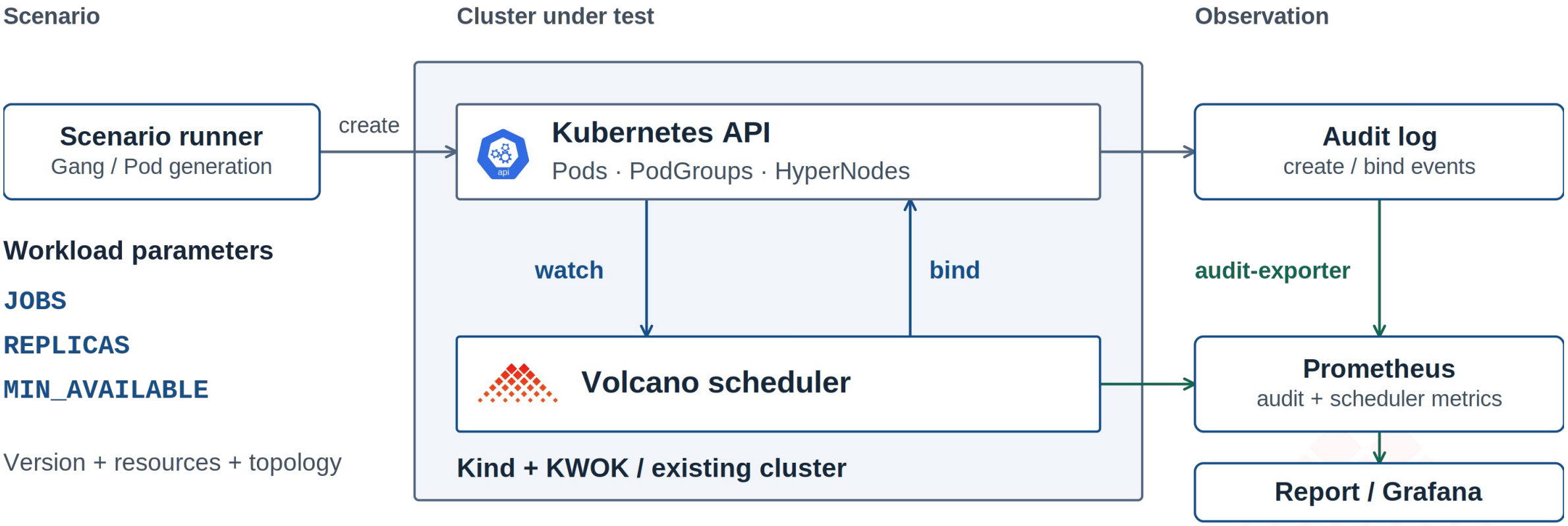
02 Multi-level gang & network topology

03 Ecosystem

04 Benchmark framework & progress

A Repeatable Scheduling Benchmark Framework

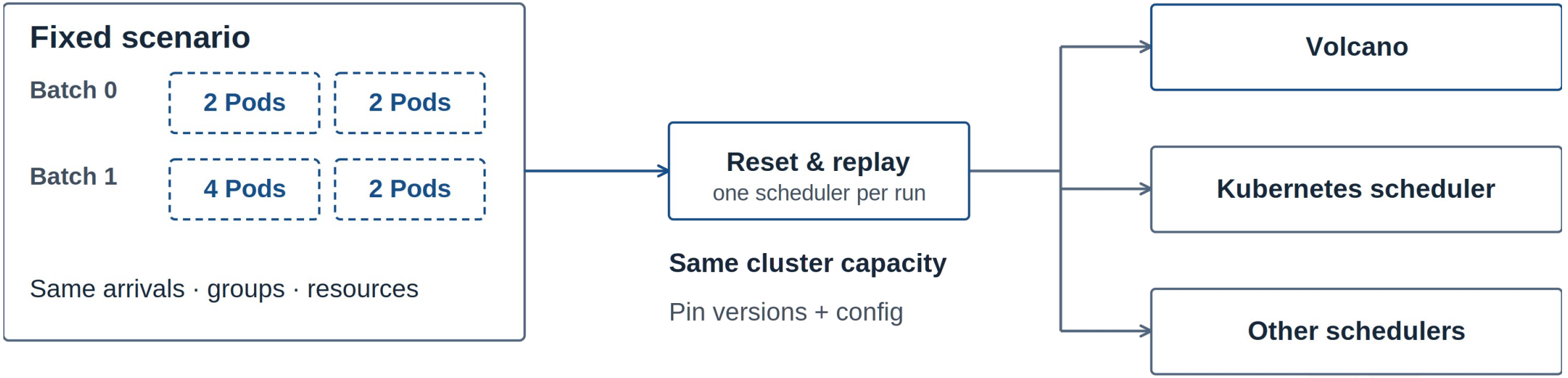
Generate scenarios, observe API events, and retain comparable measurements.



Compare Schedulers on the Same Workload

Repeat controlled runs to understand throughput, latency, and placement tradeoffs.

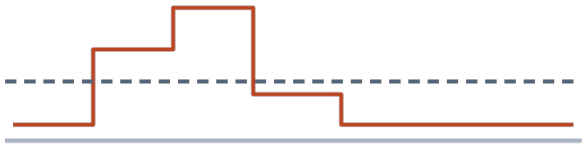
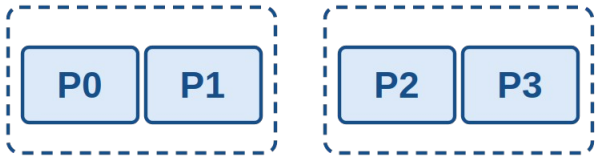
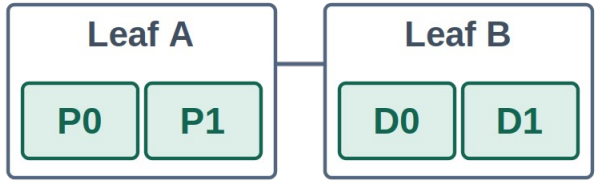
One workload, replayed



Compare latency · binding throughput · completed gangs · placement

Which Scenarios Fit Which Scheduler?

Expand tests across arrival patterns, group structure, and network topology.

Test shape	What to vary	What to observe
Bursts 	**Arrival rate / spare capacity** Warm-pool capacity requests	**Tail scheduling latency** Backlog recovery
Gang size 	**SubGroup size / fragmentation** Few large vs. many small groups	**Complete gangs placed** Binding throughput
Topology 	**Tier limits / available domains** Locality + gang + anti-affinity	**Placement correctness** Gang completion + latency

Find the best-fit scenarios and test topology-aware gang placement.

What We Will Work On Next

Next steps in group placement, controller integration, and benchmarking.

Workstream	Next step	What to validate
Group placement	**SubGroup affinity / anti-affinity** Extend the existing gang and topology model.	Cooperating groups stay local. Replicas use separate domains.
Community integration	**RBG · Kubeflow · LWS multi-group** Carry roles and replica identity into scheduling.	Controllers preserve complete groups and map placement constraints.
Benchmark validation	**Compare schedulers across scenarios** Find best-fit cases. Test topology-aware gangs.	Retain latency distributions, binding throughput, and raw events.

Volcano community



Thank you

volcano.sh

GitHub

Slack

Questions?